

Rappresentazione dell'informazione

Introduzione

Sin da piccoli siamo abituati a contare in base 10 (sistema decimale): questo vuol dire che con 10 cifre rappresentiamo tutti i numeri. Si tratta di un sistema posizionale: vuol dire che se scriviamo 45 è diverso da 54, perché anche se le cifre sono le stesse il loro valore cambia in base alla posizione. In particolare, partendo da destra, la prima cifra viene moltiplicata per 10^0 , la seconda per 10^1 , la terza per 10^2 e così via.

Questo sistema è nato in modo spontaneo: avendo 10 dita, contiamo fino a 10 (9 se contiamo lo 0) e poi ricominciamo. Un computer, che viene alimentato dalla corrente, conosce solo due stati: acceso o spento (è vero che potrebbe anche riconoscere, per esempio, 10 livelli di tensione diversi; ma è estremamente più complicato del sistema acceso/spento). Quindi “conta con due dita”, ossia in base 2 (sistema binario). Questo vuol dire che la rappresentazione dei numeri per noi e per il computer è diversa: ciò non toglie che possiamo rappresentare le stesse informazioni.

Una **rappresentazione** è un modo per descrivere un'entità. Noi rappresentiamo i numeri in base 10 (sistema decimale); il computer lo fa in base 2 (sistema binario). Un'informazione può essere rappresentata in più modi, ma il suo valore resta lo stesso.

Rappresentazione con il sistema binario

Unità di misura

Quindi un computer, avendo a disposizione due cifre 0 (spento) e 1 (acceso) conterà in base 2: 0 (0), 1 (1), 10 (2), 11 (3), 100 (4), 101 (5) e via dicendo. Come abbiamo le unità (poi decine, poi centinaia, migliaia e via dicendo) nel sistema decimale, abbiamo degli equivalenti anche nel sistema binario.

Un **bit** (binary digit) è l'unità di misura dell'informazione. Può valere solo 0 o 1. Più bit formano altre strutture:

1 nybble = 4 bit

1 byte = 8 bit

1 halfword = 16 bit

1 word = 32 bit

1 doubleword = 64 bit

1 kilo = 2^{10}

1 mega = 2^{20}

1 giga = 2^{30}

Il bit più a sinistra (la prima cifra) è detto **MSB** (*most significant bit*).

Il bit più a destra (l'ultima cifra) è detto **LSB** (*less significant bit*).

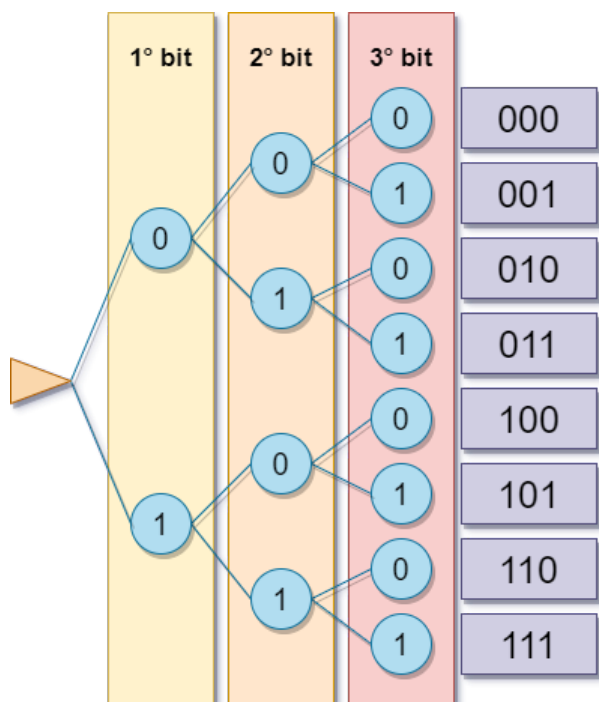
N.B.: il resto mondo scrive nybble così: *nibble*. Nybble, o l'equivalente nyble, usano la y per fare un parallelismo (inutile) con byte. Sono corretti entrambi, ma a scanso di equivoci, la forma più utilizzata è nibble.

N.B.: per l'MSB e l'LSB, basta pensare: se ho 1001 euro, qual è l'1 più importante? Sicuramente il primo, che mi indica le migliaia (1000 euro) piuttosto che l'1 delle unità (1 euro). Quindi a sinistra MSB e a destra LSB.

Informazioni rappresentabili

Ovviamente il numero di informazioni diverse che possiamo rappresentare dipende sia dalla base (vedi [qui < https://blue3141.dev/knowledge-base/sistemi-numerici-e-conversione/>](https://blue3141.dev/knowledge-base/sistemi-numerici-e-conversione/): basi con numeri grandi permettono di avere più informazioni in meno spazio. Quindi la base 2 su questo aspetto è pessima) che appunto dallo spazio (il numero di bit) che si hanno a disposizione.

Calcolare quante configurazioni diverse si possono avere con un certo numero di bit è semplice. Supponiamo di avere 3 bit: il primo può essere 0 o 1, il secondo 0 o 1, il terzo idem. Quindi possiamo scegliere in due modi il primo, in due modi il secondo, in due modi il terzo, così:



< <https://i0.wp.com/blue3141.dev/wp-content/uploads/2021/03/dd-1.png?ssl=1> >

Diagramma delle “scelte”: ogni bit può avere 2 valori, quindi ad ogni nuovo bit il diagramma raddoppia i rami.

Viene quindi naturale vedere che ogni bit aggiunto raddoppia le possibili configurazioni. Quindi, con un bit potremo rappresentare 2^1 informazioni diverse, con due $2^2 = 4$, con tre $2^3 = 8$. Visto che per trovare quante informazioni facciamo $2^{\text{numero_dei_bit}} = \text{informazioni}$, l'operazione inversa sarà $\text{numero_dei_bit} = \log_2(\text{informazioni})$.

Numero di **informazioni rappresentabili** con b bit: $i = 2^b$

Numero di **bit necessari** a rappresentare i informazioni: $b = \log_2(i)$.

N.B.: Se vi chiedono, nel sistema decimale, quanti numeri puoi creare avendo a disposizione tre posti (ossia: quanti numeri da 3 cifre ci sono), cosa rispondi? Beh: se ho massimo 3 cifre, posso contare da 0 a 999. Quindi ho 999 numeri (da 1 a 999) più uno (lo 0): in totale 1000 numeri. Che, guarda caso, è 10^3 : visto che siamo in base 10 e abbiamo a disposizione 3 posti, possiamo rappresentare 1000 informazioni (numeri) diversi. Questa cosa funziona anche per il sistema in base 2: con 5 bit, quante informazioni ho? Il numero più piccolo è 00000, 0 in base decimale, mentre il più grande 11111, ossia 31 in base decimale (vedi come [convertire](https://blue3141.dev/knowledge-base/sistemi-numerici-e-conversione/) < <https://blue3141.dev/knowledge-base/sistemi-numerici-e-conversione/> >). Quanti numeri ci sono da 0 a 31? $31 + 1 = 32 = 2^5$. Come vedete, torna tutto!

Overflow

L'**overflow** si verifica quando un'operazione di somma o sottrazione ha come risultato un numero non rappresentabile con le cifre a disposizione.

Il concetto è molto semplice: supponiamo di avere una calcolatrice che può mostrare 3 cifre sul display e fare calcoli con numeri di massimo 3 cifre. Inseriamo 199, poi premiamo +, e inseriamo 825. Il conto è banale, anche a mente conosciamo il risultato: 1024. Eppure, la calcolatrice non può farlo, perché è in grado di rappresentare numeri da 3 cifre al massimo, mentre il risultato ne ha 4. Quindi la calcolatrice va in overflow.